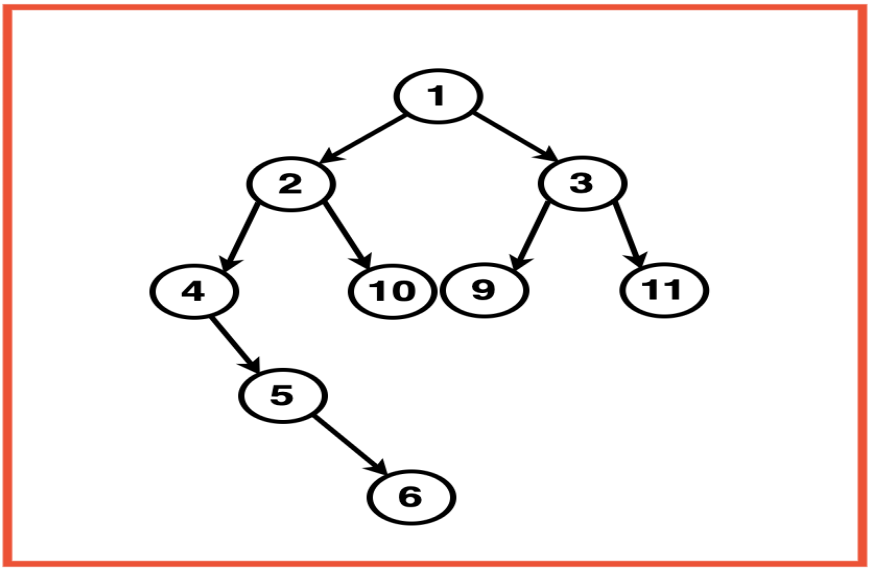
Right/Left view of binary tree

Problem Statement: Given a Binary Tree, return its right and left views.

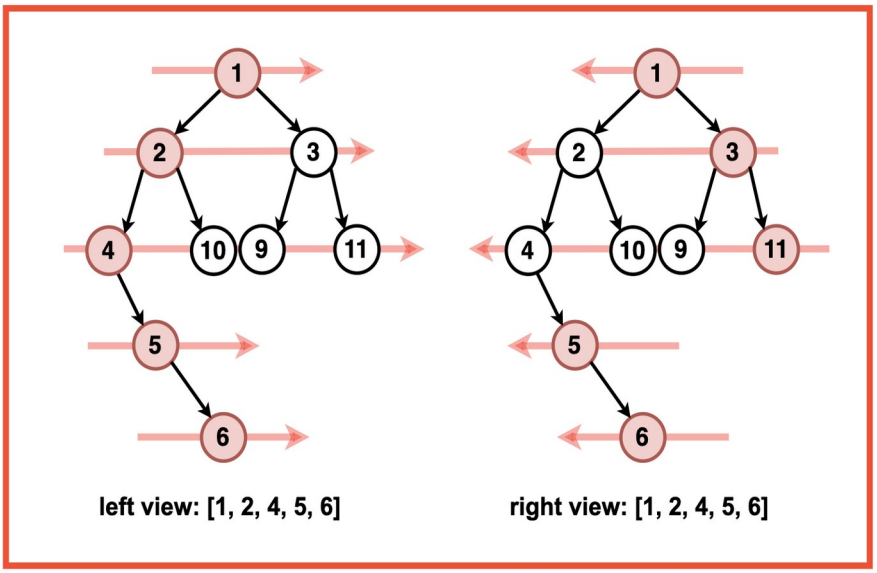
The Right View of a Binary Tree is a list of nodes that can be seen when the tree is viewed from the right side. The Left View of a Binary Tree is a list of nodes that can be seen when the tree is viewed from the left side.

Examples

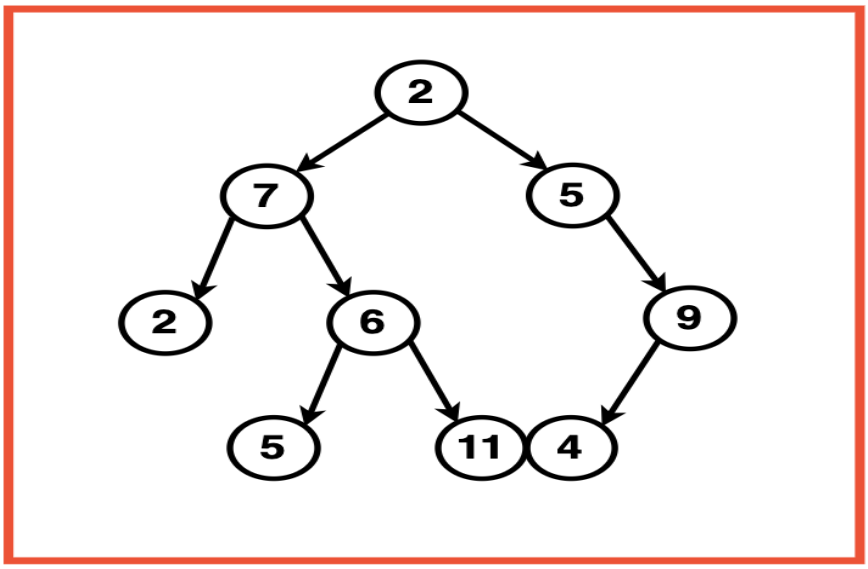
Example 1:
Input: Binary Tree: 1 2 3 4 10 9 11 -1 5 -1 -1 -1 -1 -1 -1 -1 6



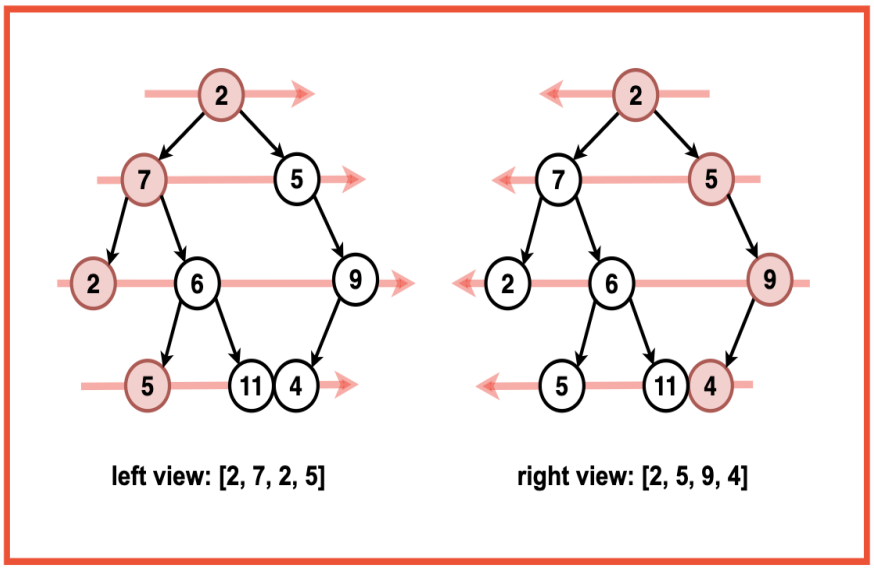
Output: Left View: [1, 2, 4, 5, 6] , Right View: [1, 3, 11, 5, 6]
Explanation:



Example 2:
Input: Binary Tree: 2 7 5 2 6 -1 9 -1 -1 5 11 4 -1



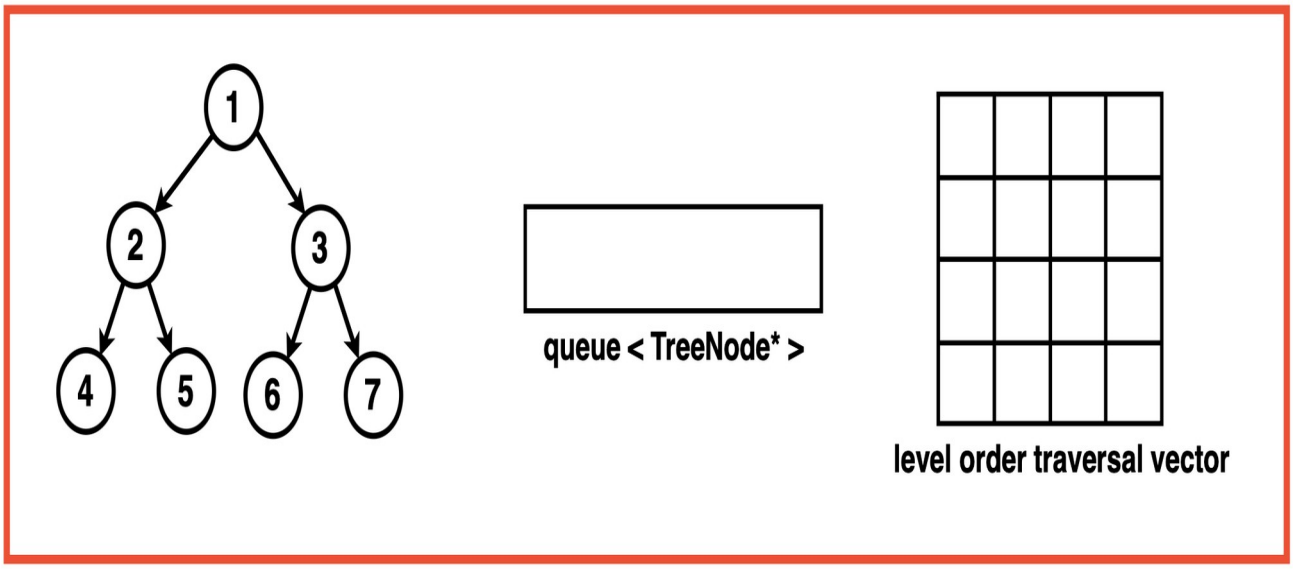
Output :`Left View Traversal:[2,7,2,5], Right View Traversal:[2,5,9,4]`
Explanation:



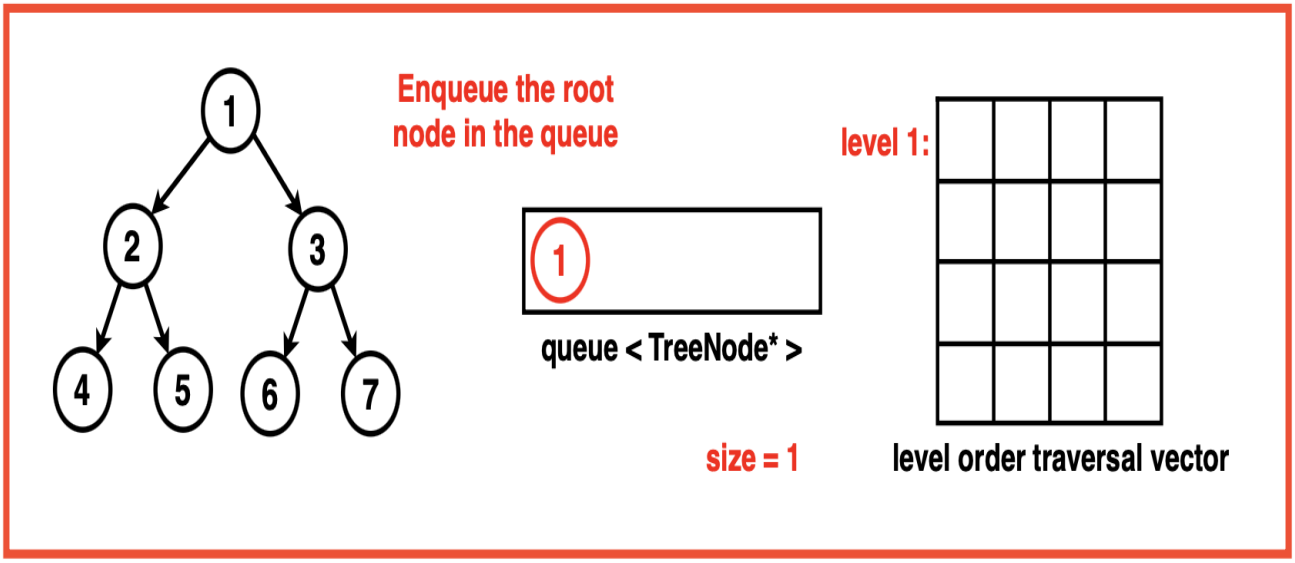
Brute Force Approach
Algorithm / Intuition

Algorithm:

Step 1: Initialise an empty queue data structure to store the nodes during traversal. Create a 2D array or a vector of a vector to store the level order traversal. If the tree is empty, return this empty 2D vector.

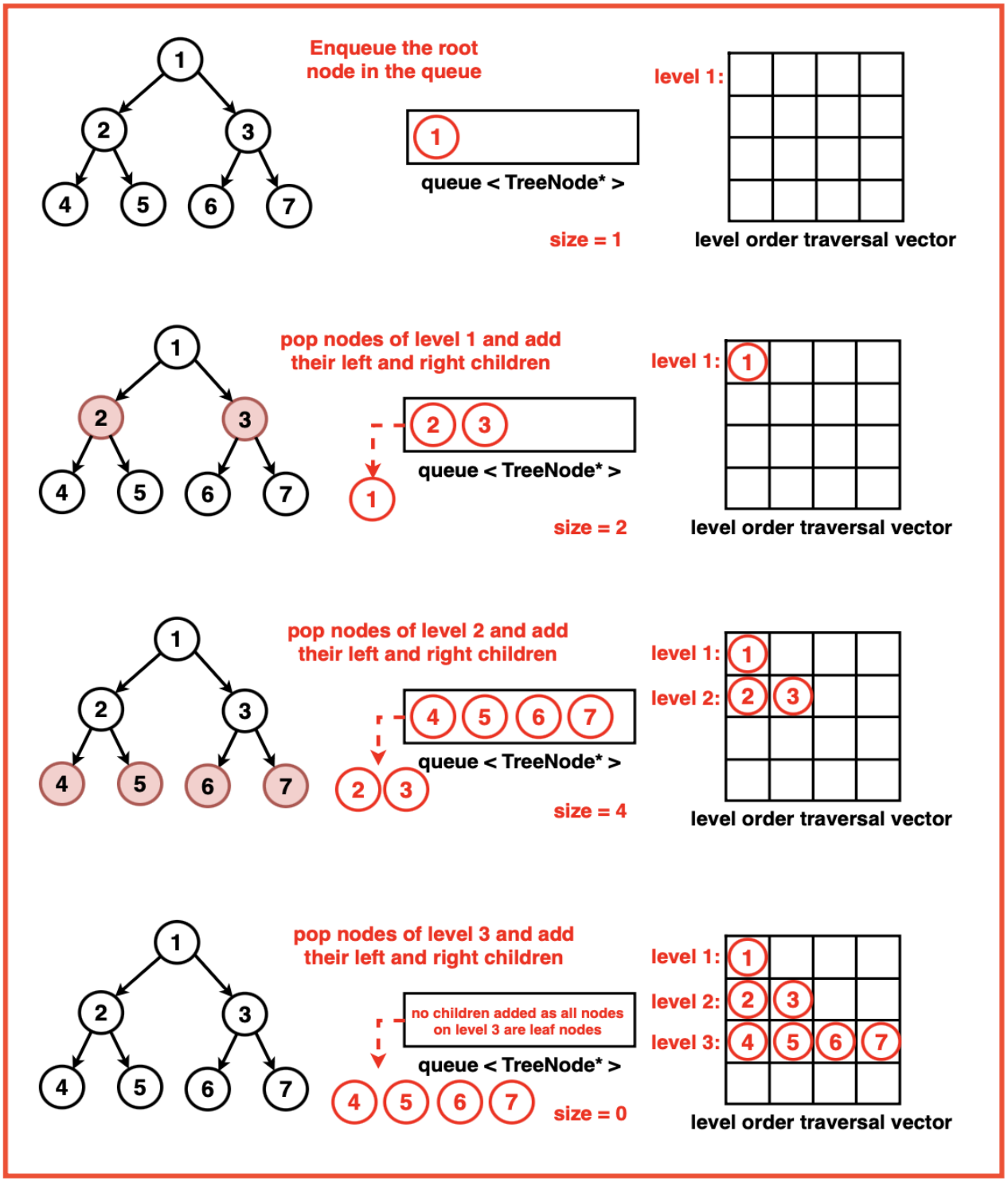


Step 2: Enqueue the root node ie. Add the root node of the binary tree to the queue.



Step 3: Iterate until the queue is empty:

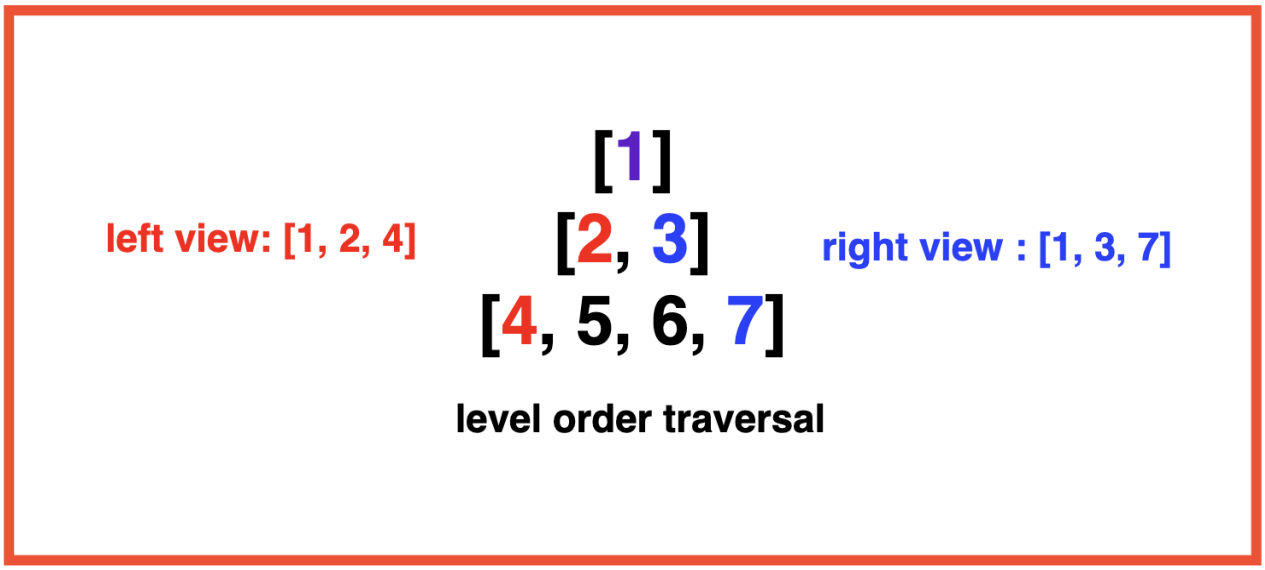
1. Get the current size of the queue. This size indicates the number of nodes at the current level.
2. Create a vector ‘level’ to store the nodes at the current level.
3. Iterate through ‘size’ number of nodes at the current level:
    1. Pop the front node from the queue.
    2. Store the node’s value in the level vector.
    3. Enqueue the left and right child nodes of the current node (if they exist) into the queue.
4. After processing all the nodes at the current level, add the ‘level’ vector to the ‘ans’ 2D vector, representing the current level.



Step 4: Once the traversal loop completes, the 'ans' 2D vector now contains the level order traversal of the binary tree. To obtain the left view and right view we use each level's vector in the 'ans' vector

Left View: For each level, extract the first element from the vector and store it in a separate array. Return this array as the final left view of the binary tree.

Right View: For each level, extract the last element from the vector and store it in a separate array. Return this array as the final right view of the binary tree.



Code

C++

```
#include <iostream>
#include <vector>
#include <set>
#include <queue>
#include <map>

using namespace std;

// Node structure for the binary tree
struct Node {
    int data;
    Node* left;
    Node* right;
    // Constructor to initialize
    // the node with a value
    Node(int val) : data(val),
        left(nullptr), right(nullptr) {}
};

class Solution {
public:
    // Function to return the
    // Right view of the binary tree
    vector<int> rightsideView(Node* root) {
        // Vector to store
        // the result
        vector<int> res;

        // Get the level order
        // traversal of the tree
        vector<vector<int>> levelTraversal = levelOrder(root);

        // Iterate through each level and
        // add the last element to the result
        for (auto level : levelTraversal) {
            res.push_back(level.back());
        }

        return res;
    }
```

```
    // Function to return the
    // Left view of the binary tree
    vector<int> leftsideView(Node* root) {
        // Vector to store the result
        vector<int> res;

        // Get the level order
        // traversal of the tree
        vector<vector<int>> levelTraversal = levelOrder(root);

        // Iterate through each level and
        // add the first element to the result
        for (auto level : levelTraversal) {
            res.push_back(level.front());
        }

        return res;
    }

private:
    // Function that returns the
    // level order traversal of a Binary tree
    vector<vector<int>> levelOrder(Node* root) {
        vector<vector<int>> ans;

        // Return an empty vector
        // if the tree is empty
        if (!root) {
            return ans;
        }

        // Use a queue to perform
        // level order traversal
        queue<Node*> q;
        q.push(root);

        while (!q.empty()) {
            int size = q.size();
            vector<int> level;

            // Process each node
            // in the current level
            for (int i = 0; i < size; i++) {
                Node* top = q.front();
                level.push_back(top->data);
                q.pop();

                // Enqueue the left
                // child if it exists
                if (top->left != NULL) {
                    q.push(top->left);
                }

                // Enqueue the right
                // child if it exists
                if (top->right != NULL) {
                    q.push(top->right);
                }
            }

            // Add the current
            // level to the result
            ans.push_back(level);
```

```
        }

        return ans;
    }
};



int main() {
    // Creating a sample binary tree
    Node* root = new Node(1);
    root->left = new Node(2);
    root->left->left = new Node(4);
    root->left->right = new Node(10);
    root->left->left->right = new Node(5);
    root->left->left->right->right = new Node(6);
    root->right = new Node(3);
    root->right->right = new Node(10);
    root->right->left = new Node(9);

    Solution solution;

        // Get the Right View traversal
    vector<int> rightView = solution.rightsideView(root);

    // Print the result for Right View
    cout << "Right View Traversal: ";
    for(auto node: rightView){
        cout << node << " ";
    }
    cout << endl;

    // Get the Left View traversal
    vector<int> leftView = solution.leftsideView(root);

    // Print the result for Left View
    cout << "Left View Traversal: ";
    for(auto node: leftView){
        cout << node << " ";
    }
    cout << endl;

    return 0;
}
--------------------------------------------------------------------------------
JAVA

import java.util.ArrayList;

import java.util.List;

// Node structure for the binary tree
class Node {
    int data;
    Node left;
    Node right;

    // Constructor to initialize
    // the node with a value
    Node(int val) {
        data = val;
        left = null;
```

```
        right = null;
    }
}

public class Solution {

    // Function to return the Right view of the binary tree
    public List<Integer> rightsideView(Node root) {
        // List to store the result
        List<Integer> res = new ArrayList<>();

        // Call the recursive function
        // to populate the right-side view
        recursionRight(root, 0, res);

        return res;
    }

    // Function to return the Left view of the binary tree
    public List<Integer> leftsideView(Node root) {
        // List to store the result
        List<Integer> res = new ArrayList<>();

        // Call the recursive function
        // to populate the left-side view
        recursionLeft(root, 0, res);

        return res;
    }

    // Recursive function to traverse the
    // binary tree and populate the left-side view
    private void recursionLeft(Node root, int level, List<Integer> res) {
        // Check if the current node is null
        if (root == null) {
            return;
        }

        // Check if the size of the result list
        // is equal to the current level
        if (res.size() == level) {
            // If equal, add the value of the
            // current node to the result list
            res.add(root.data);
        }

        // Recursively call the function for the
        // left child with an increased level
        recursionLeft(root.left, level + 1, res);

        // Recursively call the function for the
        // right child with an increased level
        recursionLeft(root.right, level + 1, res);
    }

    // Recursive function to traverse the
    // binary tree and populate the right-side view
    private void recursionRight(Node root, int level, List<Integer> res) {
        // Check if the current node is null
        if (root == null) {
            return;
        }

        // Check if the size of the result list
```

```
        // is equal to the current level
        if (res.size() == level) {
            // If equal, add the value of the
            // current node to the result list
            res.add(root.data);

            // Recursively call the function for the
            // right child with an increased level
            recursionRight(root.right, level + 1, res);

            // Recursively call the function for the
            // left child with an increased level
            recursionRight(root.left, level + 1, res);
        }
    }

    public static void main(String[] args) {
        // Creating a sample binary tree
        Node root = new Node(1);
        root.left = new Node(2);
        root.left.left = new Node(4);
        root.left.right = new Node(10);
        root.left.left.right = new Node(5);
        root.left.left.right.right = new Node(6);
        root.right = new Node(3);
        root.right.right = new Node(10);
        root.right.left = new Node(9);

        Solution solution = new Solution();

        // Get the Right View traversal
        List<Integer> rightView = solution.rightsideView(root);

        // Print the result for Right View
        System.out.print("Right View Traversal: ");
        for (int node : rightView) {
            System.out.print(node + " ");
        }
        System.out.println();

        // Get the Left View traversal
        List<Integer> leftView = solution.leftsideView(root);

        // Print the result for Left View
        System.out.print("Left View Traversal: ");
        for (int node : leftView) {
            System.out.print(node + " ");
        }
        System.out.println();
    }
}
```

Output: Right View Traversal: 1 3 10, Left View Traversal: 1 2 4 5 6

Complexity Analysis

Time Complexity: O(N) where N is the number of nodes in the binary tree. Each node of the binary tree is enqueued and dequeued exactly once, hence all nodes need to be processed and visited. Processing each node takes constant time operations which contributes to the overall linear time complexity.

Space Complexity : O(N) where N is the number of nodes in the binary tree. In the worst case, the queue has to hold all the nodes of the last level of the binary tree, the last level could at most hold N/2 nodes hence the space complexity of the queue is proportional to O(N). The resultant vector answer also stores the values of the nodes level by level and hence contains all the nodes of the tree contributing to O(N) space as well.

Optimal Approach
Algorithm / Intuition

To get the left and right view of a Binary Tree, we perform a depth-first traversal of the Binary Tree while keeping track of the level of each node. For both the left and right view, we'll ensure that only the first node encountered at each level is added to the result vector.

Algorithm for Left View

Step 1: Initialise an empty vector `res` to store the left view nodes.

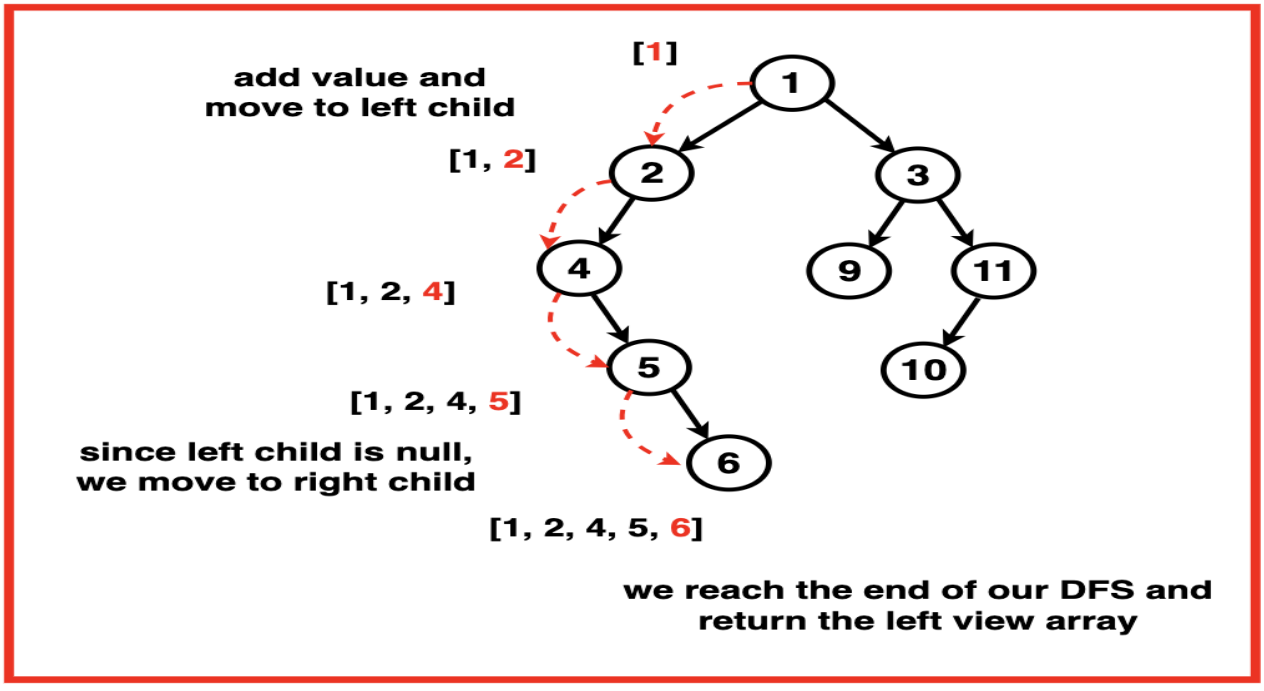
Step 2: Implement a recursive depth-first traversal of the binary tree.

Base Case: Check if the current node is null, if true, return the function as we have reached the end of that particular vertical level.

Recursive Function: The recursive function takes in arguments the current node of the Binary Tree, its current level and the result vector.

1. We check if the size of the result vector is equal to the current level.
2. If true, it means that we have not yet encountered any node at this level in the result vector. Add the value of the current node to the result vector.
3. Recursively call the function for the current nodes left then right child with an increased level ie. level + 1.
4. We call the left child first as we want to traverse the left most nodes. In cases where there is no left child, the recursion function backtracks and explores the right child.

Step 3: The recursion continues until it reaches the base case. Return the result vector at the end.



Algorithm for Right View

Step 1: Initialise an empty vector `res` to store the left view nodes.

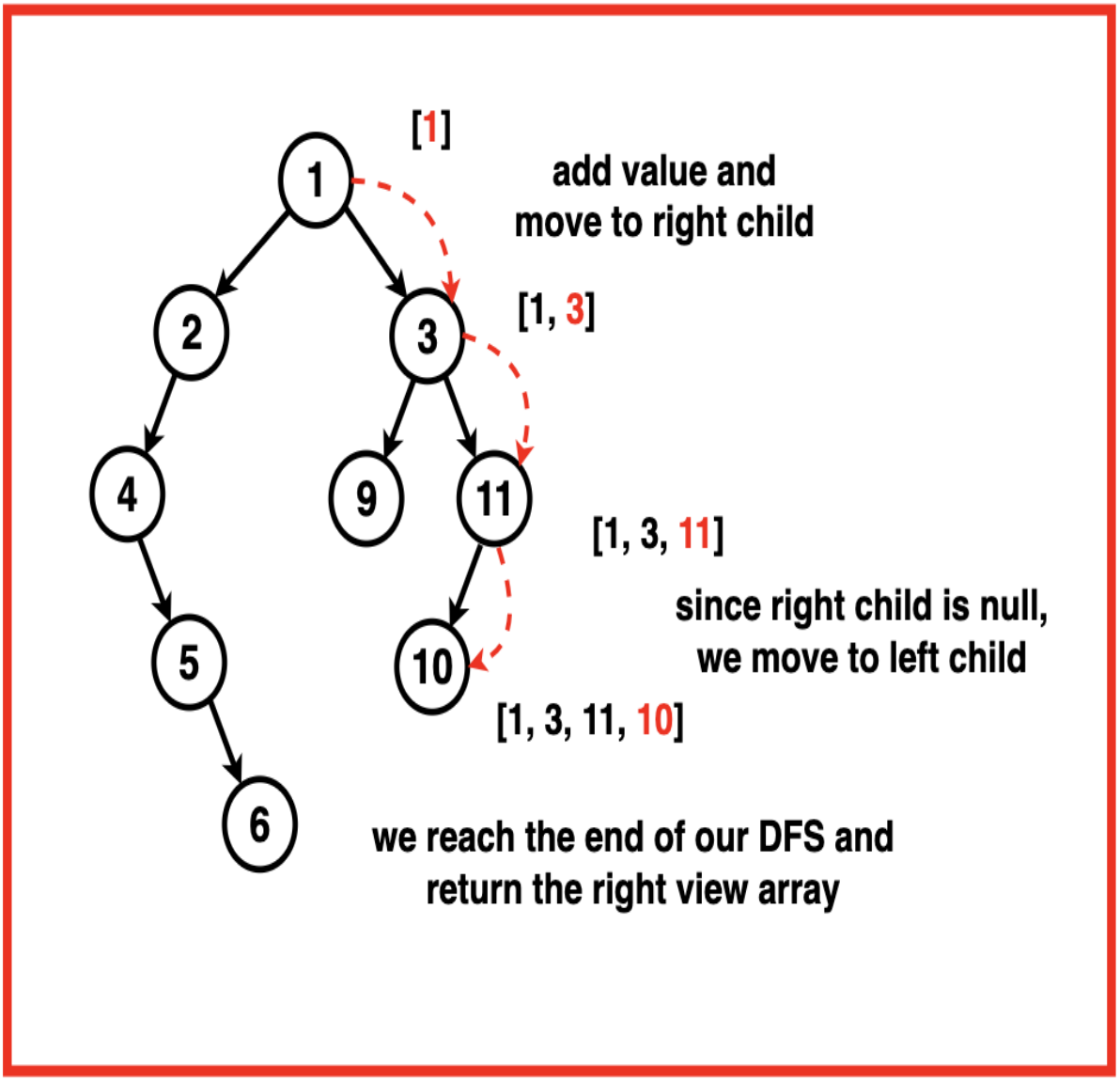
Step 2: Implement a recursive depth-first traversal of the binary tree.

Base Case: Check if the current node is null, if true, return the function as we have reached the end of that particular vertical level.

Recursive Function: The recursive function takes in arguments the current node of the Binary Tree, its current level and the result vector.

1. We check if the size of the result vector is equal to the current level.
2. If true, it means that we have not yet encountered any node at this level in the result vector. Add the value of the current node to the result vector.
3. Recursively call the function for the current nodes right then left child with an increased level ie. level + 1.
4. We call the right child first as we want to traverse the right most nodes. In cases where there is no right child, the recursion function backtracks and explores the left child.

Step 3: The recursion continues until it reaches the base case. Return the result vector at the end.



Code
C++

```
#include <iostream>
#include <vector>
#include <set>
#include <queue>
#include <map>

using namespace std;

// Node structure for the binary tree
struct Node {
    int data;
    Node* left;
    Node* right;
    // Constructor to initialize
    // the node with a value
    Node(int val) : data(val), left(nullptr), right(nullptr) {}
};

class Solution {
public:
    // Function to return the Right view of the binary tree
    vector<int> rightsideView(Node* root){
        // Vector to store the result
        vector<int> res;

        // Call the recursive function
        // to populate the right-side view
        recursionRight(root, 0, res);

        return res;
    }

    // Function to return the Left view of the binary tree
    vector<int> leftsideView(Node* root){
        // Vector to store the result
        vector<int> res;

        // Call the recursive function
        // to populate the left-side view
        recursionLeft(root, 0, res);

        return res;
    }

private:
    // Recursive function to traverse the
    // binary tree and populate the left-side view
    void recursionLeft(Node* root, int level, vector<int>& res){
        // Check if the current node is NULL
        if(root == NULL){
            return;
        }

        // Check if the size of the result vector
        // is equal to the current level
        if(res.size() == level){
            // If equal, add the value of the
            // current node to the result vector
            res.push_back(root->data);
```

```
        }

        // Recursively call the function for the
        // left child with an increased level
        recursionLeft(root->left, level + 1, res);

        // Recursively call the function for the
        // right child with an increased level
        recursionLeft(root->right, level + 1, res);
    }

    // Recursive function to traverse the
    // binary tree and populate the right-side view
    void recursionRight(Node* root, int level, vector<int> &res){
        // Check if the current node is NULL
        if(root == NULL){
            return;
        }

        // Check if the size of the result vector
        // is equal to the current level
        if(res.size() == level){
            // If equal, add the value of the
            // current node to the result vector
            res.push_back(root->data);

            // Recursively call the function for the
            // right child with an increased level
            recursionRight(root->right, level + 1, res);

            // Recursively call the function for the
            // left child with an increased level
            recursionRight(root->left, level + 1, res);
        }
    }
};



int main() {
    // Creating a sample binary tree
    Node* root = new Node(1);
    root->left = new Node(2);
    root->left->left = new Node(4);
    root->left->right = new Node(10);
    root->left->left->right = new Node(5);
    root->left->left->right->right = new Node(6);
    root->right = new Node(3);
    root->right->right = new Node(10);
    root->right->left = new Node(9);

    Solution solution;

        // Get the Right View traversal
    vector<int> rightView = solution.rightsideView(root);

    // Print the result for Right View
    cout << "Right View Traversal: ";
    for(auto node: rightView){
        cout << node << " ";
    }
    cout << endl;

    // Get the Left View traversal
```

```
    vector<int> leftView = solution.leftsideView(root);

    // Print the result for Left View
    cout << "Left View Traversal: ";
    for(auto node: leftView){
        cout << node << " ";
    }
    cout << endl;

    return 0;
}
```

JAVA

```
import java.util.ArrayList;

import java.util.List;

// Node structure for the binary tree
class Node {
    int data;
    Node left;
    Node right;

    // Constructor to initialize
    // the node with a value
    Node(int val) {
        data = val;
        left = null;
        right = null;
    }
}

public class Solution {

    // Function to return the Right view of the binary tree
    public List<Integer> rightsideView(Node root) {
        // List to store the result
        List<Integer> res = new ArrayList<>();

        // Call the recursive function
        // to populate the right-side view
        recursionRight(root, 0, res);

        return res;
    }

    // Function to return the Left view of the binary tree
    public List<Integer> leftsideView(Node root) {
        // List to store the result
        List<Integer> res = new ArrayList<>();

        // Call the recursive function
        // to populate the left-side view
        recursionLeft(root, 0, res);

        return res;
    }

    // Recursive function to traverse the
    // binary tree and populate the left-side view
    private void recursionLeft(Node root, int level, List<Integer> res) {
        // Check if the current node is null
```

```
        if (root == null) {
            return;
        }

        // Check if the size of the result list
        // is equal to the current level
        if (res.size() == level) {
            // If equal, add the value of the
            // current node to the result list
            res.add(root.data);
        }

        // Recursively call the function for the
        // left child with an increased level
        recursionLeft(root.left, level + 1, res);

        // Recursively call the function for the
        // right child with an increased level
        recursionLeft(root.right, level + 1, res);
    }

    // Recursive function to traverse the
    // binary tree and populate the right-side view
    private void recursionRight(Node root, int level, List<Integer> res) {
        // Check if the current node is null
        if (root == null) {
            return;
        }

        // Check if the size of the result list
        // is equal to the current level
        if (res.size() == level) {
            // If equal, add the value of the
            // current node to the result list
            res.add(root.data);

            // Recursively call the function for the
            // right child with an increased level
            recursionRight(root.right, level + 1, res);

            // Recursively call the function for the
            // left child with an increased level
            recursionRight(root.left, level + 1, res);
        }
    }

    public static void main(String[] args) {
        // Creating a sample binary tree
        Node root = new Node(1);
        root.left = new Node(2);
        root.left.left = new Node(4);
        root.left.right = new Node(10);
        root.left.left.right = new Node(5);
        root.left.left.right.right = new Node(6);
        root.right = new Node(3);
        root.right.right = new Node(10);
        root.right.left = new Node(9);

        Solution solution = new Solution();

        // Get the Right View traversal
        List<Integer> rightView = solution.rightsideView(root);

        // Print the result for Right View
```

```
        System.out.print("Right View Traversal: ");
        for (int node : rightView) {
            System.out.print(node + " ");
        }
        System.out.println();

        // Get the Left View traversal
        List<Integer> leftView = solution.leftsideView(root);

        // Print the result for Left View
        System.out.print("Left View Traversal: ");
        for (int node : leftView) {
            System.out.print(node + " ");
        }
        System.out.println();
    }
}
```

Output: Right View Traversal: 1 3 10, Left View Traversal: 1 2 4 5 6

Complexity Analysis

Time Complexity: O(log2N) where N is the number of nodes in the Binary Tree. This complexity arises as we travel along the height of the Binary Tree. For a balanced binary tree, the height is log2N but in the worst case when the tree is skewed, the complexity becomes O(N).

Space Complexity : O(log2N) where N is the number of nodes in the Binary Tree. This complexity arises because we store the leftmost and rightmost nodes in an additional vector. The size of this result vector is proportional to the height of the Binary Tree which will be log2N when the tree is balanced and O(N) in the worst case of a skewed tree.

1. O(H): Recursive Stack Space is used to calculate the height of the tree at each node which is proportional to the height of the tree.
2. The recursive nature of the getHeight function, which incurs space on the call stack for each recursive call until it reaches the leaf nodes or the height of the tree.